

2-HOUR SESSION

SQL Views & Stored Procedures

*Master database programming: simplify queries with views
and automate tasks with stored procedures*

Topics: Views • Stored Procedures • Parameters • Control Flow • Best Practices

Session Agenda — 2 Hours

0:00–0:10



Introduction to Database Objects

0:10–0:40



SQL Views — Concepts & Examples

0:40–0:50



Hands-on: Writing Your First View

0:50–1:20



Stored Procedures — Core Concepts

1:20–1:45



Parameters, Flow Control & Examples

1:45–2:00



Best Practices & Q&A

What is a View?

Definition: A VIEW is a virtual table based on a stored SQL SELECT query. It does NOT store data physically — it runs the query dynamically each time it is accessed.



Virtual Table

Looks and acts like a table but stores no data itself



Security Layer

Restricts access to sensitive columns or rows



Reusability

Write complex queries once, reuse them everywhere

Creating a View — Syntax & Example

Syntax

```
CREATE VIEW view_name AS  
SELECT columns FROM table  
WHERE condition;
```

Modify / Replace

```
CREATE OR REPLACE VIEW view_name AS  
SELECT new_columns FROM table;
```

Real-World Example: Employee Salary View

```
-- Create a view to show employee info without sensitive salary data  
CREATE VIEW vw_employee_public AS  
  SELECT emp_id, first_name, last_name, department, hire_date  
  FROM employees  
  WHERE status = 'ACTIVE';  
  
-- Query the view just like a table!  
SELECT * FROM vw_employee_public WHERE department = 'Sales';
```



DROP VIEW: DROP VIEW view_name;



Cannot DROP a view if other views depend on it!

Types of Views

Simple View

Based on a single table. No GROUP BY, DISTINCT, or joins. Supports INSERT/UPDATE/DELETE.

```
CREATE VIEW vw_active_emp AS
SELECT * FROM employees
WHERE status = 'ACTIVE';
```

Complex View

Uses multiple tables (JOINS), GROUP BY, aggregate functions. Generally read-only.

```
CREATE VIEW vw_dept_summary AS
SELECT d.name, COUNT(e.id) AS emp_count
FROM dept d JOIN employees e
  ON d.id = e.dept_id
GROUP BY d.name;
```

Materialized View*

Stores results physically on disk. Faster reads. Needs manual REFRESH. (*Oracle/PostgreSQL)

```
CREATE MATERIALIZED VIEW mv_sales AS
SELECT product_id, SUM(amount)
FROM orders
GROUP BY product_id;
```

Hands-On: View with JOIN & WHERE

 **Scenario: Show customer names, their orders, and order totals — only for orders above \$500**

```
-- Step 1: Understand the source tables
-- customers(id, name, email)  orders(id, cust_id, order_date, total)

-- Step 2: Create the view
CREATE VIEW vw_high_value_orders AS
  SELECT c.name          AS customer_name,
         o.id            AS order_id,
         o.order_date,
         o.total          AS order_total
  FROM customers c
  INNER JOIN orders o ON c.id = o.cust_id
  WHERE o.total > 500
  ORDER BY o.total DESC;
```

Sample Output

customer_name	order_id	order_date	order_total
Alice Johnson	1042	2024-03-15	\$1,200
Bob Smith	1087	2024-03-18	\$850
Carol Lee	1103	2024-03-20	\$670

PART 2

Stored Procedures

Precompiled blocks of SQL stored in the database that can be executed on demand — like functions for your database.



Fast Execution



Secure



Reusable



Maintainable

Stored Procedure — Basic Syntax

1

Change delimiter (MySQL)

2

CREATE PROCEDURE block

3

Body between BEGIN...END

4

CALL to execute

```
-- MySQL Syntax
DELIMITER //

CREATE PROCEDURE sp_get_employees()
BEGIN
    SELECT emp_id, first_name, last_name, department, salary
    FROM employees
    WHERE status = 'ACTIVE'
    ORDER BY last_name;
END //

DELIMITER ;

-- Execute the stored procedure
CALL sp_get_employees();
```


Stored Procedures with Parameters

IN

Passes value INTO the procedure. Most common.

```
IN dept_name VARCHAR(50)
```

OUT

Returns value FROM the procedure.

```
OUT emp_count INT
```

INOUT

Passes value in AND returns modified value.

```
INOUT salary DECIMAL(10,2)
```

```
-- SP with IN parameter: Filter by department
DELIMITER //
CREATE PROCEDURE sp_emp_by_dept(IN p_dept VARCHAR(50))
BEGIN
    SELECT emp_id, first_name, salary
    FROM employees
    WHERE department = p_dept;
END //
DELIMITER ;

-- Call with a value
CALL sp_emp_by_dept('Sales');
CALL sp_emp_by_dept('Engineering');
```

OUT Parameters — Returning Values

💡 Use OUT when you want the SP to compute a value and pass it back to the caller

```
DELIMITER //
CREATE PROCEDURE sp_count_dept_emp(
    IN  p_dept    VARCHAR(50),
    OUT p_count   INT
)
BEGIN
    SELECT COUNT(*)
    INTO   p_count
    FROM   employees
    WHERE  department = p_dept
          AND status = 'ACTIVE';
END //
DELIMITER ;

-- How to call and read OUT parameter
CALL sp_count_dept_emp('Engineering', @total);
SELECT @total AS employee_count;  -- result: 12
```

CALL with IN='Engineering'



SP counts rows → @total



SELECT @total → 12

Control Flow: IF / ELSEIF / ELSE



Syntax Template

```
IF condition THEN
    statements;
ELSEIF other_condition THEN
    statements;
ELSE
    statements;
END IF;
```



Use Case: Grade a student score

- IF score ≥ 90 → Grade A
- IF score ≥ 75 → Grade B
- IF score ≥ 60 → Grade C
- ELSE → Grade F

```
DELIMITER //
CREATE PROCEDURE sp_assign_grade(
    IN p_score INT,
    OUT p_grade VARCHAR(2)
)
BEGIN
    IF p_score  $\geq$  90 THEN
        SET p_grade = 'A';
    ELSEIF p_score  $\geq$  75 THEN
        SET p_grade = 'B';
    ELSEIF p_score  $\geq$  60 THEN
        SET p_grade = 'C';
    ELSE
        SET p_grade = 'F';
    END IF;
```

```
END //
```

```
DELIMITER ;
```

Control Flow: WHILE Loop & CURSORS

WHILE Loop

```
CREATE PROCEDURE sp_sum_1_to_n(  
    IN p_n INT,  
    OUT p_total INT  
)  
BEGIN  
    DECLARE i INT DEFAULT 1;  
    SET p_total = 0;  
    WHILE i <= p_n DO  
        SET p_total = p_total + i;  
        SET i = i + 1;  
    END WHILE;  
END //
```

```
-- CALL sp_sum_1_to_n(10, @s);  
SELECT @s; -- Output: 55
```

CURSOR (Row-by-Row)

```
DECLARE done INT DEFAULT 0;  
DECLARE v_name VARCHAR(100);  
DECLARE cur CURSOR FOR  
    SELECT name FROM employees;  
DECLARE CONTINUE HANDLER  
    FOR NOT FOUND SET done = 1;  
  
OPEN cur;  
read_loop: LOOP  
    FETCH cur INTO v_name;  
    IF done THEN  
        LEAVE read_loop;  
    END IF;
```

```
-- process v_name...  
END LOOP;
```

 **Cursor vs Set-Based:** Cursors iterate row-by-row and are slower than set-based SQL. Prefer JOINS and aggregations where possible. Use cursors only when row-by-row processing is unavoidable.

Views vs Stored Procedures — At a Glance

Feature	SQL View	Stored Procedure
Stores Data?	✗ No (virtual)	✗ No (code only)
Accepts Parameters?	✗ No	✓ Yes (IN/OUT/INOUT)
Returns Result Set?	✓ Yes (always)	✓ Yes (optional)
Control Flow (IF/LOOP)?	✗ No	✓ Yes
Can Modify Data?	Limited (simple views only)	✓ Yes (INSERT/UPDATE/DELETE)
DML Inside Definition?	✗ No	✓ Yes
Called With?	SELECT / FROM	CALL proc_name()
Best Used For	Simplified queries, security	Business logic, automation

Best Practices

Naming Conventions

Use prefixes: vw_ for views (vw_employee_summary), sp_ for procedures (sp_get_orders). Makes objects easy to identify.

Always Add Comments

Document purpose, parameters, and author. Future you (and teammates) will thank you!

Use Views for Security

Expose only needed columns via views. Never let users query raw tables with sensitive data (SSN, passwords).

Error Handling in SPs

Use DECLARE ... HANDLER to catch errors. Always handle the NOT FOUND condition when using cursors.

Prefer Set-Based Logic

Avoid cursors when a single SQL JOIN/aggregate can do the job — set-based queries are 10-100x faster.

Drop Before Recreate

Use CREATE OR REPLACE VIEW / DROP PROCEDURE IF EXISTS before recreating to avoid errors in scripts.



Class Exercise — Try It Yourself!

Exercise 1 — Create a VIEW

Task: Create a view called `vw_product_summary` that shows `product_name`, `category`, `price`, and `stock_qty` from a `products` table — only for items where `price > 100` and `stock_qty > 0`. Then write a `SELECT` to show only 'Electronics' from that view.

Exercise 2 — Create a STORED PROCEDURE

Task: Write a procedure `sp_update_salary` that accepts `emp_id` (IN) and `raise_percent` (IN). It should calculate the new salary and update the `employees` table. If `emp_id` does not exist, set an OUT parameter `p_status = 'NOT FOUND'`, otherwise set it to `'UPDATED'`.

🌟 **Bonus:** Modify Exercise 1 to create a `MATERIALIZED VIEW` (or use a summary table if MySQL). Refresh it manually and compare query times!



Time: 15–20 minutes | Work in pairs | Share your solution with the class

Session Summary

 Views simplify complex queries and add a security layer — use them for read access.

 Stored Procedures encapsulate business logic with parameters, conditionals, and loops.

 Use Views for SELECT. Use SPs for DML (INSERT/UPDATE/DELETE) and complex flows.

 Always follow naming conventions, add comments, and handle errors in SPs.

 Prefer set-based SQL over cursors for performance in production databases.